VerifyTheVector

A Formally Verified System for Checking Claims About SVG Graphics Using Lean

Project Report

Submitted by:	[Sudarshana Chaitanya B N, Rajat Narandekar, Himanshi Sarode]
Institution:	[Indian Institute of Science, Bangalore]
Supervisor:	[Prof. Siddhartha Gadgil]
Date:	August 11, 2026

Contents

Abstract

With the increasing use of AI-generated content, there is a growing need to verify claims made about visual assets such as images and vector graphics. This project, **VerifyTheVector**, presents a system designed to automatically check the correctness of claims made by AI models or human annotators about a given SVG (Scalable Vector Graphics) image. The system accepts an SVG file — written in a deliberately *restricted SVG dialect* and treated as ground truth — together with a structured set of claims describing its visual content (shapes, colours, positions, counts, spatial relationships, and chart-specific properties such as pie-slice proportions or scatter-plot clusters). When the input is a raster image rather than SVG code, an AI model (Gemini) first reconstructs an equivalent SVG in this restricted dialect, automatically assigning the unique element identifiers required for verification, before the same downstream pipeline is applied.

The SVG's underlying markup is parsed by a custom parser-combinator implementation into a strongly typed document model, and each claim — expressed in a constrained, schema-validated JSON claim language rather than free natural language — is checked against this model using **Lean 4**, a dependently typed programming language and interactive theorem prover, together with the **Mathlib** library. Geometric and logical predicates (intersection, parallelism, containment, area, spatial relations, pie-chart validity, and more) are implemented as *decidable* propositions over exact integer arithmetic, allowing each claim to be reduced to a definite, computable `PASS`, `FAIL`, or `ERROR` verdict rather than an approximate or heuristic judgement. Claims may originate from a human user, from an AI model, or from a mixture of both, and compound claims are expressed compositionally using logical combinators (`all`, `any`, `not`) over an extensible library of atomic, machine-checkable predicate types.

This programmatic, formally grounded approach allows the system to identify true, false, or unverifiable claims with high reliability, reducing dependence on manual inspection. Throughout, the system assumes that the supplied SVG source is itself structurally and semantically correct; verification is concerned solely with whether the *claims* are consistent with the *SVG's actual structure*, not with whether the SVG faithfully represents any external ground truth. The project demonstrates a practical framework for automated fact-checking of visual claims and highlights the effectiveness of formal, type-driven verification techniques — in place of purely statistical or vision-based approaches — when the underlying SVG source can be trusted.

3 Introduction

3.1 Background and Motivation

The rapid growth of AI-generated content has made visual data — images, diagrams, icons, and vector graphics — an increasingly common medium through which information is communicated. Alongside this growth, AI models and human annotators frequently generate claims describing such visual content: for instance, "the chart contains three bars, the tallest of which is blue" or "the pie chart's largest slice occupies more than half of the total area." While such claims are useful for indexing, captioning, accessibility, and dataset annotation, they are not always accurate. AI-generated descriptions in particular are prone to hallucination, where the described attributes do not correspond to the actual content of the graphic.

This problem is especially significant in domains where visual accuracy is critical:

- **Accessibility.** Screen readers and assistive technologies rely on accurate descriptions of visual elements. An incorrect claim about an SVG icon, chart, or diagram could mislead visually impaired users.
- **Content Moderation.** Automated systems that flag or approve visual content based on associated claims need a mechanism to verify that those claims are trustworthy before acting on them.
- **Automated Quality Assurance (QA).** In pipelines where AI systems generate or caption large volumes of graphical content (for example, auto-generated charts and infographics), manually verifying every claim is infeasible. A scalable, automated verification mechanism is necessary.
- **Dataset Reliability.** Machine learning datasets that pair diagrams with descriptive labels are only as trustworthy as the accuracy of those labels. Programmatic claim verification helps maintain dataset integrity.

Unlike raster images, **SVG (Scalable Vector Graphics)** files present a unique opportunity for verification: because SVGs are represented as structured, machine-readable markup (an XML-based description of shapes, paths, colours, and positions) rather than raw pixel data, it is possible to analyse their *exact* underlying structure rather than relying on approximate visual interpretation. This makes SVGs particularly well suited to **formal, logic-based verification**, in contrast to the purely statistical or heuristic approaches typically used in raster image analysis.

This project exploits that property by defining a small, *restricted dialect* of SVG (Section 5) that constrains geometry to integer coordinates and a fixed set of element types, and by using **Lean 4** — a dependently typed language and proof assistant, backed by the **Mathlib** library — to represent SVG documents as strongly typed data and to check claims against them as *decidable propositions*. This allows verification to move beyond approximate pattern matching toward a deterministic, exhaustively checkable process.

3.2 Problem Statement

Given an SVG file and a set of claims describing its visual content, there is currently no lightweight, automated, and formally grounded method to verify whether these claims accurately reflect the structure of the SVG. Existing verification approaches either:

- rely on manual human review, which is slow and does not scale; or
- use vision-based models operating on rendered pixel data, which can be computationally expensive, imprecise, and unable to *guarantee* structural correctness.

There is a need for a system that can take an SVG and an associated set of claims, decompose each claim into checkable components, and systematically verify each component against the SVG's actual structure using a reliable, code-driven method — without needing to reinterpret the SVG's visual rendering from scratch each time, and without depending on the approximate judgement of a vision model for claims that are, in principle, exactly decidable.

3.3 Objectives

The primary objectives of this project are:

1. To design a restricted, verification-friendly SVG dialect that LLMs (or users) can reliably generate, in which every claim-relevant element carries a unique identifier and all geometry is expressed as exact integers.
2. To define a structured, schema-validated claim language capable of expressing geometric relations, style properties, textual content, chart-specific semantics (pie charts, scatter clusters), numeric expressions, and logical combinations thereof.
3. To implement a parser (in Lean 4) that converts raw SVG markup into a strongly typed document model.
4. To implement a verifier (in Lean 4) that evaluates each claim against the parsed document as a decidable proposition, producing a definite `PASS`, `FAIL`, or `ERROR` verdict.
5. To support claim acquisition from multiple sources — manual (user-authored), AI-generated (via Gemini), or a mixture of both.
6. To support both direct SVG input and raster image input, the latter via an AI-driven conversion step that produces a dialect-conformant SVG and an accompanying claim set.
7. To evaluate the system's correctness and reliability across a range of SVGs and claim types.

3.4 Scope and Limitations

Scope.

- The system operates on SVG markup conforming to the project's restricted dialect (Section 6.2) as the primary source of ground truth.
- Claims covered include geometric relations (intersection, parallelism, relative position, containment), style properties (fill, stroke, colour equality), textual content, general shape properties (convexity, regularity, use of curves), numeric expressions over area/perimeter/attributes, and chart-specific semantics for pie charts and scatter plots.

- Verification logic is implemented in **Lean 4** with the **Mathlib** library, using decidable instances wherever possible.
- The system is deployable as a static site (e.g., via GitHub Pages) with the Lean-based verifier compiled to an executable invoked over parsed SVG and claims input.

Limitations.

- The system **assumes the input SVG is structurally and semantically correct**; it does not validate whether the SVG itself accurately represents any external ground truth (e.g., a real-world object or a "true" dataset). Verification is limited strictly to checking claims *against the SVG*, not the SVG's own fidelity to reality.
- Only the nine element types permitted by the restricted dialect (`svg`, `line`, `rect`, `circle`, `ellipse`, `path`, `polyline`, `polygon`, `text`) are supported. Grouping/structural constructs (`g`, `defs`, `use`, `clipPath`, `mask`), gradients, filters, embedded images, and transforms are explicitly disallowed and are treated as generation errors rather than being approximately interpreted.
- All geometry must be expressed as integers; SVGs using decimal coordinates, percentages, or CSS length units on geometric attributes fall outside the supported dialect (a `viewBox` fallback exists only for the root canvas dimensions).
- Claims that cannot be expressed in the fixed claim schema (e.g., highly subjective or abstract claims such as "the image looks cheerful") are not verified; they are instead routed to an `unclaimable` list with a stated reason, rather than being forced into an ill-fitting claim type.
- The system does not currently handle animated SVGs or SVGs relying on embedded scripting (e.g. SMIL animation or JavaScript-driven elements).
- Verification performance on very large or deeply nested SVG structures has not been extensively benchmarked and is discussed further as a limitation in the evaluation.

5 System Overview

5.1 High-Level Architecture

The system is designed to be deployable as a web-based application (e.g. via **GitHub Pages**), allowing users to interact with the verifier without local installation, while the verification core itself runs as a compiled Lean executable. At a high level, the architecture consists of four major stages:

1. **Input Stage** — accepts either raw SVG markup or a raster image (PNG/JPEG).
2. **Normalisation Stage** — ensures the SVG conforms to the project's restricted dialect, with every claim-relevant element carrying a unique, stable `id` as specified in the project documentation (`docs/svg_dialect.md`). For image input, this stage is performed automatically by an AI model.
3. **Claim Acquisition Stage** — gathers the claims to be verified: manually from the user, automatically from an AI model (Gemini), or as a mixture of both.
4. **Verification Stage** — parses the normalised SVG into a typed document model and checks each claim against it using Lean-based decidable predicates, producing a final verdict.

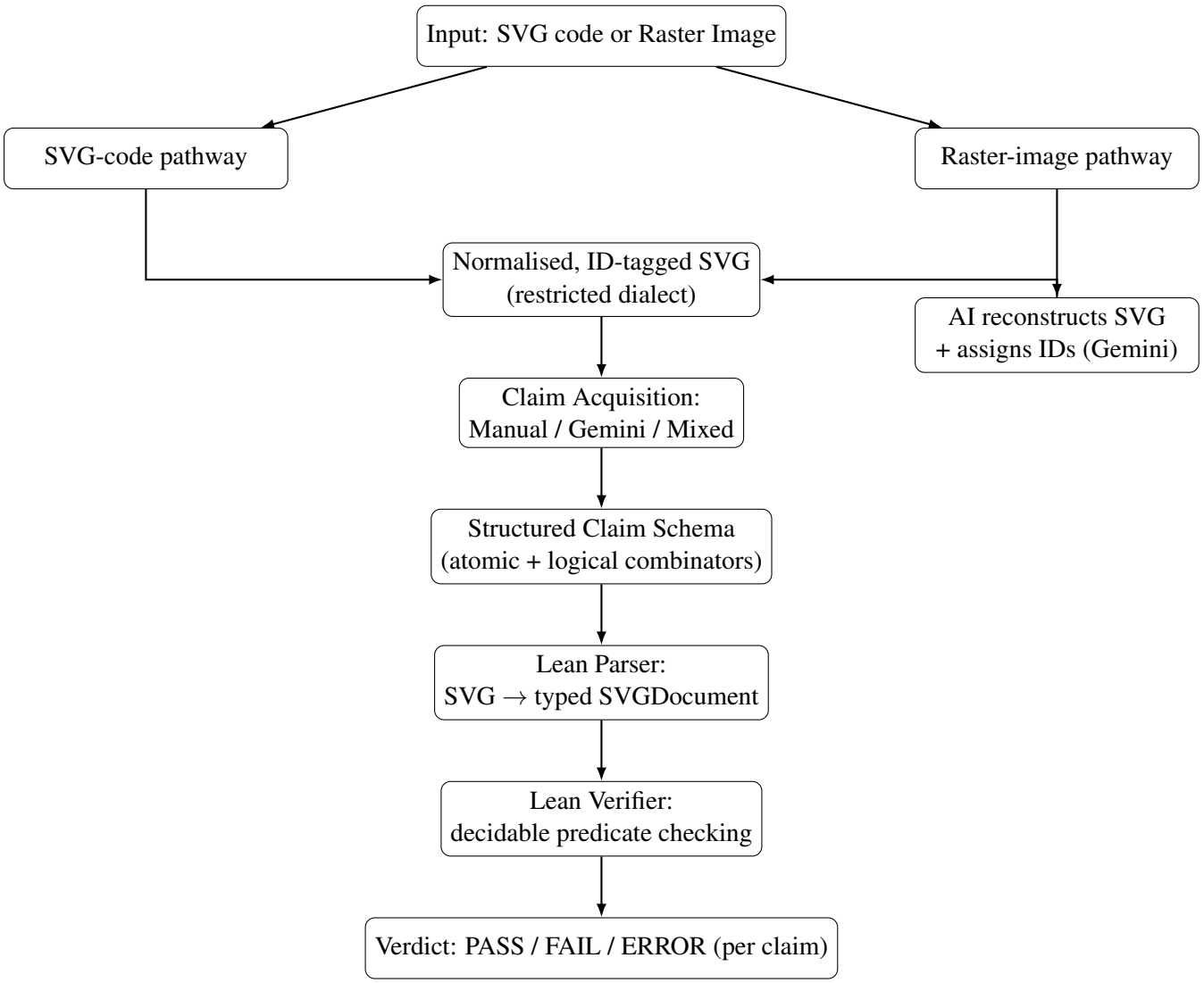

Figure 1: High-level architecture: two input pathways converge on a normalised, ID-tagged SVG before entering the shared claim-acquisition and Lean-based verification pipeline.

5.2 Two Input Pathways: Raster Image vs. Raw SVG Code

(a) Raw SVG Code Pathway. When the user directly supplies an SVG file, it is expected to already follow the conventions of the restricted dialect defined in `docs/svg_dialect.md` — most notably, that every element referenced by a claim carries a unique `id`, that all geometry is integer-valued, and that only the nine supported element types are used. These conventions ensure that every element referenced in a claim can be unambiguously located and matched within the SVG's parsed structure.

(b) Raster Image Pathway. When the user instead supplies a standard raster image (PNG/JPEG), the system does not attempt to verify claims directly against pixel data. Instead, an AI conversion step (Gemini):

1. analyses the image and reconstructs its content as an equivalent SVG representation in the restricted dialect;

2. automatically assigns the required unique IDs and permitted style attributes (`fill`, `stroke`, `stroke-width`) to each generated element; and

3. produces an accompanying claims JSON object (and any `unclaimable` observations) describing the image's content, validated against `schemas/llm_output.schema.json`.

This ensures that regardless of the input type, the system always arrives at the same normalised artefact — a dialect-conformant, ID-tagged SVG together with a schema-valid claims list — before verification begins, allowing both pathways to be treated identically downstream.

5.3 Convergence into a Common Verification Pipeline

Once an SVG has been obtained — supplied directly or generated from a raster image — both pathways converge into a single, shared verification pipeline:

1. **Claim Acquisition.** Claims about the SVG's content can be introduced in one of three ways:

   - **Manual** — the user writes the claims directly, following the JSON claim schema.
   - **AI-generated (Gemini)** — the Gemini model analyses the SVG/image and automatically produces a schema-conformant claims list.
   - **Mixed** — user-provided and AI-generated claims are combined, enabling cross-checking between human- and AI-authored descriptions.

2. **Structured Decomposition.** Regardless of source, claims are expressed directly in the constrained claim schema (Section 6.3) rather than as free text. Compound statements are built compositionally from atomic claim types using logical combinators (`all`, `any`, `not`), so decomposition happens at claim-authoring time.

3. **Parsing.** The normalised SVG markup is parsed by a Lean parser-combinator implementation into a strongly typed `SVGDocument`.

4. **Structural Matching via Lean.** Each claim is looked up against the parsed document by element `id`, and the relevant decidable geometric, style, textual, or numeric predicate is evaluated.

5. **Verdict Generation.** Each claim receives one of three outcomes — `PASS`, `FAIL`, or `ERROR` (the last reserved for unverifiable claims, e.g. a missing ID or a type mismatch) — and the full list of per-claim verdicts is returned to the user.

By funnelling both image-based and SVG-based inputs into the same normalised representation, the system guarantees consistent verification logic regardless of how the visual content was originally supplied, while the upfront dialect/ID normalisation step ensures that the Lean-based matching stage always operates on a predictable, well-typed input.

6 Methodology / System Design

6.1 Input Handling

The system's entry point is a JSON payload conforming to `llm_output.schema.json`, containing exactly three top-level fields: `svg` (the raw SVG markup as a string), `claims` (an array of structured claim objects), and `unclaimable` (observations the LLM noticed but could not express in the claim schema). When the input is a raster image rather than direct SVG code, the AI conversion step described in Section 5.3 produces this same {`svg, claims, unclaimable`} shape, so downstream processing is entirely agnostic to the original input modality.

6.2 The Restricted SVG Dialect

Rather than attempting to support arbitrary SVG, the project defines a **restricted SVG dialect** (documented in `docs/svg_dialect.md`) that any generated or submitted SVG must follow. Key constraints include:

- Only nine element types are permitted: `svg, line, rect, circle, ellipse, path, polyline, polygon, text`. Structural/grouping and styling constructs — `g`, `defs`, `use`, `symbol`, `marker`, `clipPath`, `mask`, `style`, `foreignObject`, gradients, filters, and embedded images — are disallowed.
- All geometric values (`x, y, cx, cy, r, rx, ry, width, height`, points/-path coordinates) must be **integers** — no decimals, scientific notation, percentages, or CSS units on geometry. This is what allows the Lean side to use exact integer arithmetic and decidable equality rather than floating-point approximation.
- **Every element referenced by a claim must carry a unique `id`.** IDs are the join key between the claims JSON and the parsed SVG DOM; stable, semantic IDs are recommended (e.g. `bar_q1`, `slice_large`, `cluster_left`).
- **Transforms are disallowed.** Elements must carry final, absolute coordinates rather than relying on `transform="translate(...)"` or coordinate-system-changing groups.
- **Styling** is optional and limited to `fill`, `stroke`, and `stroke-width`; CSS stylesheets, inherited group styles, and classes are not relied upon for verification-critical meaning.
- **Path data** is restricted to the command set `M, L, H, V, C, S, Q, T, A, Z` with absolute, uppercase, space/comma-separated numeric coordinates.
- Domain-specific conventions are defined for common chart types:
  - *Pie slices* must be encoded as a path following the pattern `M center → L rimStart → A rx ry rotation largeArcFlag sweepFlag rimEnd → L center → Z`, enabling claims such as `validPieSlice` and `validPieChart`.
  - *Scatter clusters* are encoded as point-circles grouped by non-intersecting cluster circles, enabling claims such as `pointsInCircle` and `scatterClusterCount`.

  - *Bar ordering* is expressed compositionally via `all` plus pairwise `greaterThan` comparisons over bar heights, rather than via a dedicated "ordering" claim type.

- Any observation the AI cannot express within this schema must be reported via the `unclaimable` list (with an `observation`, a `reason`, and optionally a `suggestedClaim`) rather than being forced into an ill-fitting claim type.

Treating dialect violations as *generation errors* — to be caught before verification — rather than *missing verifier features* is a deliberate design choice: it keeps the Lean-side type system small, closed, and exhaustively decidable, at the cost of requiring the SVG producer (human or AI) to conform to the dialect.

6.3 Claim Schema

Claims are not free text; they are structured JSON objects validated against a JSON Schema (`schemas/llm_output.schema.json`) defining over thirty discriminated claim types, grouped into the following families:

Family	**Example claim types**
Geometric relations	`intersects`, `isParallel`, `sameCenter`, `hasRelation` (`above`, `leftOf`, `overlaps`, `contains`, `sameWidth`, `tallerThan`, etc.)
Style / appearance	`sameColor`, `hasFill`, `hasStroke`, `hasStrokeWidth`, `isFilled`, `isStroked`, `hasNoFill`, `hasNoStroke`, `sameFill`, `sameStroke`, `sameStrokeWidth`
Text	`textEquals`
General shape properties	`hasProperty` with values such as `closed`, `convex`, `regular`, `usesBezier`, `usesOnlyStraightEdges`, `nonzeroArea`, `nonemptyText`
Pie-chart semantics	`isPieSlice`, `isValidPieChart`, `pieChartSliceCount`, `isMajoritySlice`, `isMinorSlice`, `isHalfSlice`, `isDominantSlice`, `isMinoritySlice`, `allPieSlicesEqual`, `isBinaryPie`, `binaryPieHasLargeDominant`, `pieSliceLargerThan`, `pieSliceSmallerThan`, `pieSliceCount`
Scatter-plot semantics	`pointsInCircle`, `scatterClusterCount`
Numeric expressions	`equalTo`, `greaterThan` over composable `expr` terms: `attr`, `area`, `perimeter`, `add`, `mul`, integer literals

Family	Example claim types
Logical combinators	`all`, `any`, `not` — arbitrary boolean composition of the above

This schema *is* the claim-decomposition mechanism: rather than an NLP model freely generating natural-language claims that must later be parsed, the AI (or user) is constrained at generation time to emit claims already broken into atomic, machine-checkable predicates. Compound statements — e.g. "bar A is taller than bar B and does not cross the threshold line" — are expressed directly using `all`/`not` combinators over atomic claims, so decomposition happens at the claim-authoring stage rather than as a separate downstream NLP step.

6.4 SVG Parsing in Lean

A hand-written parser combinator implementation (`SVGParser.lean`, built on `Std.Internal.Parsec` reads the raw SVG string and produces a strongly typed `SVGDocument`:

Listing 1: Core document and element types (SVGElement.lean)

```
structure SVGStyle where
  fill : Option String
  stroke : Option String
  strokeWidth : Option Nat
deriving Repr, DecidableEq

inductive SVGElement where
  | lineseg (id : String) (style : SVGStyle) (shape : LineSeg)
  | rect (id : String) (style : SVGStyle) (shape : Rect)
  | circle (id : String) (style : SVGStyle) (shape : Circle_SVG)
  | ellipse (id : String) (style : SVGStyle) (shape : Ellipse)
  | path (id : String) (style : SVGStyle) (shape : PathSVG)
  | polyline (id : String) (style : SVGStyle) (shape : Polyline)
  | polygon (id : String) (style : SVGStyle) (shape : Polygon)
  | text (id : String) (style : SVGStyle) (shape : Text_SVG)
deriving Repr, DecidableEq

structure SVGDocument where
  width : Nat
  height : Nat
  elements : List SVGElement
deriving Repr, DecidableEq
```

Each `SVGElement` case carries its `id`, an `SVGStyle` (optional `fill`, `stroke`, `strokeWidth`), and a shape payload typed to that geometry. Separate modules under `SVGTypes/` — `Point.lean`, `Rect.lean`, `Circle.lean`, `Path.lean`, `Polygon.lean`, `Polyline.lean`, `Triangle.lean`, `LineSeg.lean`, `Ellipse.lean`, `Text.lean`, `beziercurve.lean`, and `Relations.lean` — define these geometric types along with typeclasses such as `HasArea`, `HasPerimeter`, and `Intersects`, and spatial relation predicates (`isParallel`, `above`, `leftOf`, containment, etc.), many of which are proved `Decidable` so that Lean can compute a definite answer rather than approximate one:

Listing 2: Example: viewport containment as a decidable proposition (SVGElement.lean)

```
def pointInViewport (p : Point) (doc : SVGDocument) : Prop :=
  0 <= p.x /\ p.x <= (doc.width : Int) /\
  0 <= p.y /\ p.y <= (doc.height : Int)

instance instDecidablePointInViewport (p : Point) (doc : SVGDocument) :
    Decidable (pointInViewport p doc) := by
  simp [pointInViewport]; infer_instance
```

Because the dialect fixes all coordinates to integers, geometric predicates (intersection, containment, area, parallelism) are defined and checked using exact integer arithmetic and Lean's `Decidable` typeclass machinery. This is the central benefit of using Lean over a general-purpose scripting language: the correctness of the geometric definitions can, in principle, be backed by proof rather than by testing alone.

6.5 Claim Verification

The claims JSON is decoded into a matching inductive `Claim` type (`SVGVerifier.lean`) via a hand-rolled JSON parser. Verification proceeds through a single dispatch function:

Listing 3: Verification result type and top-level dispatch (SVGVerifier.lean)

```
inductive VerifyResult where
  | pass (msg : String)
  | fail (msg : String)
  | err (msg : String)

partial def verifyClaim (doc : SVGDocument) (claim : Claim) : VerifyResult
    :=
  -- one case per claim constructor: looks up element(s) by id,
  -- pattern-matches into the concrete shape type, and invokes the
  -- relevant decidable predicate, typeclass instance, or expression
  -- evaluator (evalExpr)
  sorry -- (full pattern match omitted for brevity)

def verifyClaims (doc : SVGDocument) (claims : List Claim) : List
    VerifyResult :=
  claims.map (verifyClaim doc)
```

For each claim, `verifyClaim`:

1. looks up the referenced element `id`(s) in `doc.elements`;
2. pattern-matches the elements into their concrete shape type (e.g. extracting a `Rect` from a `.rect` constructor);
3. invokes the relevant decidable predicate, typeclass instance, or numeric expression evaluator (`evalExpr`); and
4. wraps the result into one of three outcomes: `pass`, `fail`, or `err`.

The `err` case is significant: it distinguishes a claim that is *false* from a claim that is *unverifiable as stated* (e.g. referencing a non-existent `id`, or applying a rectangle-only property to a circle) — a distinction a purely rule-based or purely LLM-based checker would tend to

blur. Logical combinators (`all`, `any`, `not`) recurse over `verifyClaim` and fold child results according to standard boolean semantics. `verifyClaims` runs this over the full claim list, producing an ordered list of per-claim verdicts, which the CLI entry point formats and prints as `PASS`/`FAIL`/`ERROR` per claim index.

6.6 Assumption of SVG Ground Truth

As stated in the abstract, the system assumes that the supplied (or AI-reconstructed) SVG is itself an accurate, correctly formed representation. The verifier's role is strictly to check whether the **claims** are consistent with the **SVG's actual structure** — not whether the SVG faithfully represents any external reality (e.g. a photographed object, or a "true" underlying dataset for a chart). This scopes the verification problem to a closed, fully decidable one: given a fixed, well-typed SVG document, every claim expressible in the schema reduces to a computable predicate over that document.

7 Implementation

7.1 Technology Stack

Component	Technology
Verification core	Lean 4 (`leanprover/lean4:v4.29.1`), built with **Lake**
Mathematical library	**Mathlib** (`v4.29.1`) — supplies decidability instances and integer/numeric infrastructure
SVG parsing	Hand-written parser combinators using `Std.Internal.Parsec` / `Std.Internal.Parsec.String`
Claim parsing	Hand-written JSON decoder (Lean's `Json` type), conceptually validated against `schemas/llm_output.schema.json`
Claim/SVG generation (image pathway)	Gemini (vision-capable LLM), prompted to emit dialect-conformant SVG and schema-conformant claims
Continuous Integration	GitHub Actions (`.github/workflows/lean_action_ci.yml`) running the Lean build on every push
Deployment target	Static site hosting (e.g. GitHub Pages) fronting the compiled Lean executable

7.2 Module Breakdown

Listing 4: Project directory structure

```
VerifyTheVector-main/
|-- Main.lean # CLI entry point: reads SVG + claims files, prints verdicts
|-- VerifyTheVector.lean # library root import
|-- VerifyTheVector/
| |-- SVGParser.lean # parser combinators: SVG string -> SVGDocument
| |-- SVGVerifier.lean # Claim type, JSON decoding, verifyClaim / verifyClaims
| |-- Basic.lean
| `-- SVGTypes/
| |-- Point.lean, Rect.lean, Circle.lean, Ellipse.lean
| |-- LineSeg.lean, Polygon.lean, Polyline.lean, Triangle.lean
| |-- Path.lean, beziercurve.lean # path & Bezier geometry, pie-slice extraction
| |-- Text.lean
| |-- Relations.lean # spatial relation predicates
| `-- SVGElement.lean # SVGElement union, SVGDocument, viewport checks
|-- schemas/llm_output.schema.json # JSON Schema for {svg, claims, unclaimable}
```

```
|-- docs/svg_dialect.md # restricted SVG dialect specification
|-- claims.json, test_chart.svg # worked example
`-- lakefile.toml, lean-toolchain # build configuration (Lean 4.29.1 +
   Mathlib)
```

“‘latex

7.3 Worked Example

This example illustrates the complete verification pipeline, beginning with natural language claims and ending with formally verified results. The only stages involving an LLM are the conversion of natural language into the project’s structured claim schema (and, when applicable, raster image to SVG conversion). All verification is subsequently performed deterministically in Lean.

Step 1: Input

Suppose the user provides the following SVG together with two natural language claims.

Listing 5: Input SVG

```svg
<svg width="200" height="200" id="chart_root">
  <line id="axis_x"
       x1="10" y1="150"
       x2="190" y2="150"
       stroke="black"/>

  <rect id="bar_A"
       x="30" y="50"
       width="40"
       height="100"
       fill="blue"/>

  <rect id="bar_B"
       x="90" y="80"
       width="40"
       height="70"
       fill="green"/>
</svg>
```

Natural language claims:

1. Bar A is taller than Bar B.
2. The area of Bar A is 4000 square units.

If the original input were a raster image (PNG/JPEG), Gemini would first reconstruct an equivalent SVG in the restricted dialect before the remaining steps are performed.

Step 2: Natural Language → JSON Claims (LLM)

The natural language claims cannot be verified directly. Gemini is prompted to translate them into the structured JSON claim schema defined by the project.

Natural Language Claims

⇓

Gemini

⇓

Schema-valid JSON Claims

The generated JSON is

Listing 6: Generated claims.json

```
{
  "claims": [
    {
      "type": "greaterThan",
      "left": {
        "type": "attr",
        "id": "bar_A",
        "attr": "height"
      },
      "right": {
        "type": "attr",
        "id": "bar_B",
        "attr": "height"
      }
    },
    {
      "type": "equalTo",
      "left": {
        "type": "area",
        "id": "bar_A"
      },
      "right": {
        "type": "num",
        "value": 4000
      }
    }
  ]
}
```

At this stage, the LLM's responsibility is complete. It only translates the user's claims into the project's machine-readable schema and does not determine whether the claims are true.

Step 3: SVG Parsing (Lean)

The SVG parser converts the XML document into a strongly typed `SVGDocument`.

Listing 7: Parsed SVGDocument

```
SVGDocument
 width = 200
 height = 200
 elements =
[
  line "axis_x",
  rect "bar_A",
```

```
  rect "bar_B"
]
```

Internally, the rectangle `bar_A` is represented as

```
SVGElement.rect
  "bar_A"
  { fill := some "blue" }
  {
    x := 30,
    y := 50,
    width := 40,
    height := 100
  }
```

Step 4: JSON Parsing (Lean)

The generated JSON is decoded into the corresponding Lean datatype.

The first claim becomes

```
Claim.greaterThan
  (Expr.attr "bar_A" "height")
  (Expr.attr "bar_B" "height")
```

The second claim becomes

```
Claim.equalTo
  (Expr.area "bar_A")
  (Expr.num 4000)
```

At this point, both the SVG and the claims have been converted into strongly typed Lean objects.

Step 5: Claim Verification

Claim 1: Bar A is taller than Bar B The verifier evaluates the expressions

$$\texttt{height(bar\_A)} \quad \text{and} \quad \texttt{height(bar\_B)}.$$

Looking up the corresponding rectangles in the document gives

$$\texttt{height(bar\_A)} = 100, \qquad \texttt{height(bar\_B)} = 70.$$

Lean therefore evaluates

$$100 > 70,$$

which is true, so the claim receives the verdict

$$\boxed{\texttt{PASS}}.$$

Claim 2: The area of Bar A is 4000 The verifier evaluates

$$\texttt{area(bar\_A)} = \texttt{width} \times \texttt{height} = 40 \times 100 = 4000.$$

It then compares

$$4000 = 4000,$$

which is again true, producing

$$\boxed{\texttt{PASS}}.$$

Step 6: Final Output

Running

```
lake exe verifythevector test_chart.svg claims.json
```

produces

```
Claim 1 : PASS
Claim 2 : PASS
```

Summary of the Verification Pipeline

Natural Language Claims
↓
Gemini
↓
JSON Claim Schema

SVG Code (or Raster Image)
↓
Raster Image → **Gemini** → SVG (if required)
↓
Lean SVG Parser
↓
Typed `SVGDocument`
↓
Lean JSON Decoder
↓
Typed `Claim` Objects
↓
Decidable Predicate Evaluation
↓
`PASS` / `FAIL` / `ERROR`

This example demonstrates the complete end-to-end workflow. Gemini is used only for converting images into SVG (when necessary) and translating natural language claims into the project's structured JSON schema. Once these inputs are available, every subsequent stage—from parsing to claim evaluation—is performed entirely within Lean using formally defined, decidable predicates. "'

7.4 Key Algorithms

- **Parser combinators** for XML-like SVG tokenisation and attribute extraction, built compositionally from primitive parsers (`parseName`, `parseNat`, `parseInt`) up to full element and path-command parsers (e.g. `parseMove`, `parseLine`).
- **Typeclass-based decidability.** Geometric predicates (`Intersects`, `HasArea`, `HasPerimeter`) are defined once as typeclasses and instantiated per shape, letting `verifyClaim` invoke generic decidable checks (e.g. `verifyIntersectsTyped`) regardless of which concrete shape is involved.
- **Pie-slice reconstruction.** `pathToPieSlice` reverse-engineers a `PieSlice` (centre, radius, start/end angle) from the constrained $\texttt{M} \to \texttt{L} \to \texttt{A} \to \texttt{L} \to \texttt{Z}$ path pattern mandated by the dialect, enabling higher-level pie-chart claims (`isValidPieChart`, `isMajoritySlice`, `binaryPieHasLargeDominant`) to be checked.
- **Expression evaluation.** `evalExpr` recursively evaluates composable numeric expressions (`attr`, `area`, `perimeter`, `add`, `mul`, integer literal) against the document, supporting claims such as "the area of A equals the area of B plus 200."

7.5 Challenges Faced

- Reproducing SVG geometry — including arcs and Bézier curves — with **exact integer arithmetic** decidable in Lean, rather than floating-point approximation, required careful type design (`beziercurve.lean`, `Path.lean`) and constraining the dialect to integer-only coordinates from the outset.
- Balancing **expressiveness against decidability** in the claim schema: every claim type had to correspond to something Lean could decide outright, which is why claims outside the schema are routed to `unclaimable` rather than approximated.
- Distinguishing a **false claim** from an **unverifiable claim** (missing id, type mismatch) required a three-valued `VerifyResult` (`pass`/`fail`/`err`) rather than a simple boolean.